

Heterogeneous CPU-GPU Implementation of Collision Detection for Gaming

Report Summarizing Design Activities - 70B

By:

Camron Tuffner-Lyons
Davis Cheung,
JJ Marr
Victor Do

Electrical & Computer Engineering
Capstone Design Project
Toronto Metropolitan University
2025

Acknowledgements

Dr. Gul N Khan, Professor
BEng, MSc, PhD, PEng

Certification of Authorship

Written by Camron Tuffner-Lyons, Davis Cheung, JJ Marr, and Victor Do.

We hereby certify that this report is of our own creation. any assistance received or sources used have been properly acknowledged and cited. We further certify that this report complies with the academic integrity policy of Toronto Metropolitan University.

Signed By:

Cameron Tuffner-Lyons

10-04-2025

Victor Do

10-04-2025

Davis Cheung

10-04-2025

JJ Marr

10-04-2025

Table of Contents

Acknowledgements	1
Certification of Authorship	2
Table of Contents	3
Abstract	5
Introduction and Background	6
Objectives	7
Partitioning the Workload	7
Developing the Algorithm	7
Building the Application	7
Assessing the Performance	7
Theory and Design	8
Introduction	8
Development Tools	8
Characteristics of CPUs and GPUs	9
Smoothed Particle Hydrodynamics (SPH)	10
Choice of Collision Detection Algorithms	12
Broad-Phase Algorithms	12
Narrow-Phase Algorithms	13
Broad and Narrow Phase Algorithms	13
Choice of Sorting Algorithms	14
Purpose	14
Bitonic Sort	14
Radix Sort	14
Counting Sort	15
Our Final Choice	15
Evaluation Metrics and Testing Methodology	16
GPU-Only Prototype for SPH Simulation	17
Design and Implementation	17
Rendering and Visualization	18
Performance Analysis of GPU-Only Prototype	19
Particle-Based Subdivision Design	20
Particle Structure and Initialization	21
SPH Simulation Parameters	22
Simulation Hyperparameters	22
Fluid Parameters	23
CPU-GPU Data Transfer Design	23
Feature Implementations	23
Dynamic CPU-GPU Device Partitioning	23
Unified Physics Kernel	24
Platform Specific Simulation Resolution Scaling	24
Level Designs	24
Level 1: Tavern Training	24

Level 2: Carnival Chaos.....	25
Level 3: Lava Laboratory.....	25
Level 4: Sliding Door Puzzle	26
Level 5: Planet Annihilation	27
Level 6: Casting a Sword	28
Level 7: Drink Delivery	29
Levels In Development:	29
Sandbox Level	30
Alternative Designs.....	30
Density-based Subdivision Design.....	30
Implementation of Density-based Subdivision	30
Measurements and Procedure	32
Performance Measurements and Analysis	33
Performance Overview	33
Wavefront Performance.....	36
Analysis	37
Conclusion	38
References	39
Appendix	41
Appendix A	41
Table A-1: Average Frame Time (ms) for Various Fluid Simulation Test Scenarios With Respect To Particles in The Fluid Simulation.....	41

Abstract

In the realm of real-time interactive simulations and gaming, fluid dynamics represent a substantial computational challenge, particularly when high levels of visual fidelity and physical accuracy are required. Collision detection plays a critical role in these simulations, often consuming a significant portion of processing time. This project proposes a heterogeneous CPU-GPU implementation designed to accelerate collision detection tasks within Smoothed Particle Hydrodynamics (SPH) fluid simulations. By distributing the computational workload between the CPU and GPU based on particle size, we aim to achieve more efficient resource utilization and improve overall simulation performance.

Our approach focuses on the implementation of a Particle-Based Subdivision algorithm, which groups fluid particles by size and assigns larger groups to the GPU and smaller groups to the CPU. This method directly addresses inefficiencies present in GPU-only implementations, where a mismatch in particle size can cause suboptimal broad-phase collision grid sizing and increased unnecessary physics computations. Using Unity as the development platform, with compute shaders written in High-Level Shader Language (HLSL) and CPU logic in C#, we created a custom simulation engine capable of executing real-time, visually engaging simulations with support for a wide variety of fluid and solid materials.

Multiple broad-phase and narrow-phase collision detection algorithms were investigated, with uniform spatial subdivision and bounding sphere methods ultimately selected due to their parallelizability and compatibility with GPU computation. Additionally, sorting algorithms such as bitonic sort, radix sort, and counting sort were investigated, with counting sort selected for its superior scalability and minimal memory overhead in GPU contexts.

Performance evaluation was conducted across ten simulation scenarios with varying particle counts and fluid types, both in GPU-only and CPU-GPU configurations. Results confirmed the effectiveness of our approach: simulations featuring fluids with significantly different particle sizes saw up to 80% improvement in frame time using the heterogeneous method. Conversely, simulations with uniform particle sizes experienced little to no improvement, validating the hypothesis that heterogeneous computation is most beneficial in mixed-particle environments. This study not only highlights the feasibility and advantages of CPU-GPU workload partitioning in fluid simulation but also lays the groundwork for further optimization techniques within real-time applications and gaming.

Introduction and Background

The objective of this project was to create a heterogeneous approach to collision detection by leveraging a combination of CPU and GPU resources. This strategy aimed to enhance computational performance and efficiency by utilizing idle CPU or GPU resources in any collision detection algorithm. Given the flexibility of the topic, we selected a specific application-focused problem within this field, centering our efforts on collision detection algorithms tailored to fluid simulations.

We targeted the Lagrangian Smoothed-Particle Hydrodynamics (SPH) method, a simulation approach where collisions occurred through interactions between individual fluid particles. This method was chosen because it provided a well-defined, discrete model for narrow-phase collisions compared to Eulerian fluid simulations. Moreover, SPH simulations were known for their computational intensity, which often limited their scalability [1]. Our approach sought to mitigate these challenges, making SPH simulations more practical for large-scale applications.

To achieve this, we developed a number of methods for workload distribution of the SPH fluid simulation into CPU and GPU workgroups. Our first method, *Density-Based Subdivision*, attempted to split particle groups into high and low density groups for parallel computing across both CPU and GPU devices. This approach was unsuccessful, leading to the development of our new approach, *Particle-Based Subdivision*, which optimized both broad phase and narrow phase collision checks by moving the simulation of fluids with large particle sizes to the CPU. By partitioning the workload into small particle fluids and large particle fluids and assigning them to the GPU and CPU, respectively, we aimed to determine whether the performance improvements from optimizing GPU broad phase collision detection could outweigh the additional latency introduced by heterogeneous device coordination. Alongside this workload distribution technique, throughout the process of developing the fluid simulation engine we implemented various kernel level optimizations to improve scalability for mobile platforms.

Our project was developed in Unity, utilizing High-Level Shader Language (HLSL) for GPU-side kernels and C# for scripting. HLSL served as the primary language for GPU computations, while C# and its extensions (e.g., DLLs) were used for tasks such as buffer initialization, memory coordination, and scheduling kernel dispatches to both CPU and GPU. This combination presented challenges in synchronizing CPU and GPU kernels and memory buffers across languages but provided valuable experience relevant to modern game development. Unity also offered an effective platform for visualizing and showcasing the results through an interactive demonstration.

Objectives

The primary goal of the capstone design project was to develop an application capable of utilizing heterogeneous CPU-GPU computing for collision detection. This section outlines the specific objectives required to design, implement, and evaluate the solution.

Partitioning the Workload

The initial objective involved analyzing CPU and GPU characteristics and applying knowledge of collision detection algorithms to identify the most suitable scenario or algorithm(s). The focus was on achieving an efficient and straightforward division of the workload between the CPU and GPU. This workload split aimed to ensure that both CPU and GPU could perform collision detection in parallel, maximizing resource utilization.

Developing the Algorithm

The second objective was to create new algorithms based on the chosen scenario and algorithm(s) from the first objective. These algorithms were required to clearly define the workload distribution between CPU and GPU, detailing each kernel's purpose and specifying where data transfers would occur. The proposed algorithms needed to demonstrate theoretical performance improvements over a GPU-only implementation for the same collision detection scenario.

Building the Application

The third objective was to design an application that implemented the CPU-GPU collision detection algorithms in a real-time, interactive environment. The application had to effectively execute the collision detection scenario, incorporate the developed algorithms, and allow users to interact with the simulation dynamically.

Assessing the Performance

The final objective involved creating a testing methodology to evaluate and compare performance across implementations (e.g., GPU-only vs. CPU-GPU). The methodology needed to ensure consistent and repeatable measurements while remaining independent of specific implementations. It also had to be integrated into the application to facilitate ongoing performance analysis and iterative improvements. The capstone project sought to achieve all these objectives, with a strong emphasis on algorithm design to optimize performance. The subsequent sections will provide a detailed overview of the methods employed to meet these objectives and the resulting design, implementation, and performance evaluation

Theory and Design

Introduction

This section focuses on the theoretical background, analysis and design of a computational workload distribution between the CPU and GPU in a real-time fluid simulation engine using Smoothed Particle Hydrodynamics (SPH). The section begins by discussing the selection of Unity as the development platform and HLSL as the GPU computing framework. It then examines the fundamental differences between CPUs and GPUs, highlighting their strengths and limitations in handling parallel processing. Various algorithms for collision detection and sorting are analyzed with respect to their applicability in SPH simulations, as well as their computational efficiency and parallelism. A major focus was initially placed on the implementation of a density-based subdivision technique, which dynamically assigned high-density regions to the CPU while allowing the GPU to handle more parallelizable tasks. This strategic workload split aimed to enhance simulation speed, minimize wavefront execution delays, and maximize hardware utilization. Our ultimate guiding design principles when developing were to create an engine that was powerful, optimized, easy to implement and to extend. This section also details the levels that were designed to highlight the simulation's feature set while also being enjoyable to play.

Development Tools

After careful consideration, Unity emerged as the ideal choice for this project. Because it lacks a built-in fluid simulation system, Unity provides an opportunity to address this gap by delivering a fully functional and optimized solution. Its use of C# streamlines development and enhances cross-platform compatibility. Unity's forward rendering pipeline and smaller project footprint make it particularly efficient and well-suited for our specific needs, as the project does not require the advanced photorealistic features offered by engines like Unreal. Furthermore, Unity's extensive documentation, academic support, and user-friendly ecosystem facilitate development and prototyping. Overall, Unity strikes the right balance between cost, performance, and flexibility, making it the most suitable platform for creating a custom fluid simulation tailored to our requirements [2].

In terms of GPU computing frameworks, High-Level Shader Language (HLSL) stands out as the preferred choice over OpenCL and CUDA for development within Unity. HLSL integrates seamlessly with Unity's rendering pipeline and offers cross-platform compatibility. Unlike the more general-purpose frameworks OpenCL and CUDA, HLSL is specifically designed for GPU programming in game engines like Unity. This makes it the most efficient and intuitive option for implementing shader-based particle simulations. Unity also supports cross-compilation of HLSL, enabling deployment to multiple platforms such as Vulkan, OpenGL and WebGPU for mobile devices or web browsers. This aligns with our goals of reaching broader platform support, ensuring the simulation runs efficiently across a wide range of devices, from desktops to embedded systems, without the need for platform-specific code modifications. Overall, HLSL provides the necessary flexibility, performance, and compatibility to meet the requirements of this project within Unity's framework [3].

Characteristics of CPUs and GPUs

GPUs are designed for parallel processing, employing a Single Instruction, Multiple Threads (SIMT) architecture that contrasts with the sequential execution focus of CPUs. While CPUs execute instructions sequentially and use multicore designs to handle multiple threads by assigning distinct instruction sets to each core, GPUs optimize parallelism by combining Single Instruction, Multiple Data (SIMD) principles with multicore processing. On GPUs, tasks are divided into threads, organized into groups called warps (or wavefronts), which are processed in parallel by GPU cores. Conditional execution is managed through masking, ensuring that inactive threads do not disrupt the processor state. [4]

GPU performance is heavily dependent on maintaining wave coherency, where all threads in a warp execute the same instruction simultaneously. Divergent threads reduce efficiency by causing other threads within the warp to perform unnecessary operations. Memory architecture is another critical factor. GPUs use device memory for high-bandwidth operations, shared memory for low-latency communication between threads in close proximity, and registers for thread-specific operations. Efficient memory management can mitigate register pressure, a GPU-specific bottleneck caused by the limited availability of registers for threads within a warp. [4]

To enhance GPU performance, algorithms often prioritize shared memory over device memory for inter-thread communication and manage register allocation carefully to reduce bottlenecks. These optimization strategies are essential for achieving high performance in heterogeneous CPU-GPU systems and parallel processing applications. [4]

Smoothed Particle Hydrodynamics (SPH)

The choice to develop a fluid simulation was due to it being a practical and relevant application of collision detection for gaming. The choice of a Lagrangian (particle-based) SPH simulation in particular was because a fully particle-based method is ideal for modeling complex, dynamic free-surface flows such as splashes and breaking waves. Unlike grid-based methods, SPH's grid-less nature simplifies handling irregular geometry boundaries and dynamic topologies, with the capability for adaptive resolution where detail is most needed. The Fluid Toy also supports the simulation of multiple types of fluids and solids, showcasing SPH's versatility in modeling viscous fluids, multiphase flows, and deformable solids [5].

Another factor behind choosing a particle-based simulation is that collisions are not particularly well-defined for non-particle-based simulations (Eulerian sims). Consequently, a non-particle-based simulation would not involve many forms of discrete collision detection and would make it more difficult to find areas in collision detection to improve to provide meaningful performance improvements. As a result, our project would not be particularly viable if we chose a non-particle-based simulation to base it on.

Smoothed Particle Hydrodynamics (SPH) is a Lagrangian technique used to simulate fluid flows and other continuous media by breaking the fluid into individual particles, each representing properties like mass, velocity, and pressure. These particles interact through smoothing kernel functions derived from Navier-Stokes equations [1], which makes SPH highly adaptable to complex geometries and free surfaces.

Our project relies on several GPU kernels, which execute multiple times each frame to calculate the fluid behavior: **SpawnParticles**, **ExternalForces** (currently implementing gravity), **UpdateSpatialHash** (part 1 of the broad-phase spatial subdivision algorithm), **ReorderArray and CopybackArray** (part 2 of the spatial subdivision algorithm), **CalculateDensities, UnifiedForceCalculation, UpdatePositions** (to write back to the buffers) and finally **UpdateStates** (for particle-type changes). The density and physics kernels use the results from the broad-phase collision detection to compute properties for neighboring particles.

Key considerations include the data dependencies between stages (e.g., density calculations feed into pressure, which then feeds into viscosity). This dependency means threads performing computations must wait for prior calculations to complete in order to synchronize. Additionally, the CPU-GPU boundary adds complexity, as particle data must be merged at boundary points. The remainder of this section elaborates on SPH and the equations used to model individual particles in the simulation.

In SPH, fluids are represented as spherical particles carrying attributes like mass, velocity, and pressure. These particles interact based on their relative positions, and physical effects are calculated using a smoothing kernel over the surrounding neighborhood. SPH excels in simulating free-surface flows, astrophysical phenomena, and other situations where grid-based methods are less efficient.

Density of a particle in SPH is determined by summing the contributions of neighboring particles weighted by a kernel function, which is a smooth function that decreases with distance. The density at a particle is given by:

$$\rho_i = \sum_j m_j W(\mathbf{r}_i - \mathbf{r}_j, h) \quad (1)$$

where m_j is the mass of particle $\mathbf{r}_i$, $\mathbf{r}_j$ are the positions of particles i and j , and h is the smoothing length controlling the interaction range. The kernel W ensures only nearby particles influence one another, smoothing the local density distribution.

Pressure is calculated using an equation of state, often simplified to the ideal gas law, which relates pressure to density. The pressure force between particles is derived from the negative gradient of the pressure field:

$$\mathbf{F}_i^{pressure} = - \sum_j m_j \left(\frac{P_i + P_j}{2\rho_j^2} + \frac{c_s^2}{\rho_j} \right) \nabla W(\mathbf{r}_i - \mathbf{r}_j, h) \quad (2)$$

where P_i, P_j are the pressures of particles i and j , ρ_j is the density of particle j , and c_s is the speed of sound in the medium. This accounts for fluid compressibility, with its gradient driving the pressure forces.

Viscosity in SPH is represented through a dissipative term that depends on the velocity difference between particles. An artificial viscosity term prevents inter-particle instabilities and models viscous forces:

$$\mathbf{F}_i^{viscosity} = - \sum_j m_j \left(\frac{v_i + v_j}{\rho_j} \right) \nabla W(\mathbf{r}_i - \mathbf{r}_j, h) \quad (3)$$

This term acts more strongly when particles exhibit relative motion and scales with the velocity difference.

Temperature was relatively straightforward. Using a simple conversion of the heat exchange equation for an SPH approach with particle-particle interactions, and appending the smoothing kernel, we get:

$$\Delta T = \sum_j \alpha_j \left(\frac{T_i + T_j}{\rho_j} \right) \nabla W(\mathbf{r}_i - \mathbf{r}_j, h) \quad (4)$$

where T_i, T_j are the temperatures of particles i and j , and α_j is the thermal diffusivity of j , simplified to a constant for each fluid in the simulation.

Friction follows a similar equation to viscosity, except it is skipped for fluids, only existing on powders, and forgoes density and mass, being entirely dependent on the relative speeds of neighbouring particles, and a frictional coefficient. It is given by:

$$\mathbf{F}_i^{friction} = - \sum_j \mu_j (v_i - v_j) \nabla W(\mathbf{r}_i - \mathbf{r}_j, h) \quad (5)$$

Where μ_j is the frictional coefficient of particle j . In addition to this force, particles with friction, or *powders*, will experience an overall velocity damping force which decays the velocity magnitude by one of two different scalars, one which applies when the particle is in motion to mimic air resistance and a stronger decay force when the particle is at rest, to help it form ‘pile’ like structures, mimicking static friction. Powders will have pressure and viscosity forces of zero to save on unnecessary calculations.

Finally, the **velocity update** for a particle combines all acting forces. The rate of change of velocity is given by:

$$\frac{dv_i}{dt} = \frac{1}{\rho_i} (\mathbf{F}_i^{pressure} + \mathbf{F}_i^{viscosity} + \mathbf{F}_i^{external} + \mathbf{F}_i^{friction}) \quad (6)$$

This velocity update drives the evolution of particle motion, simulating fluid dynamics over time as particles interact under these forces.

Choice of Collision Detection Algorithms

Collision detection is a fundamental aspect of simulations, particularly in SPH (Smoothed Particle Hydrodynamics) environments. Broad-phase and narrow-phase algorithms are employed to efficiently detect and manage collisions. This literature review examines several broad-phase and narrow-phase algorithms, assessing their strengths, limitations, and suitability for real-time simulations.

Broad-Phase Algorithms

The information in this section regarding broad-phase algorithms comes from [6]. Bounding Volume Hierarchies (BVH) use nested bounding volumes to exclude non-colliding objects efficiently. They are particularly effective for irregular or rotated shapes. However, in dynamic environments, updating the hierarchy incurs high computational costs, and their sequential nature limits GPU performance, which can hinder real-time interactivity.

Uniform Spatial Subdivision partitions space into a fixed grid where each grid cell has an id created with a hashing function, and objects are assigned to specific cells according to their positions. This method is highly parallelizable and well-suited for SPH simulations, where particles are uniform spheres. Nevertheless, it struggles in unevenly distributed scenes, as densely populated cells create bottlenecks while empty cells waste resources. Proper cell size selection is crucial to balance computational efficiency and accuracy [7].

Quad-Trees (or Oct-Trees in 3D) subdivide space into smaller regions, optimizing collision detection by restricting checks to localized areas. While effective for spatial organization, they suffer similar issues to BVHs in dynamic scenes, where imbalanced trees and frequent restructuring degrade efficiency. Their recursive nature also presents challenges for GPU parallelization.

Narrow-Phase Algorithms

The Gilbert-Johnson-Keerthi (GJK) algorithm determines collisions between convex shapes by iteratively constructing a Minkowski difference. While highly precise and efficient for discrete convex objects, its iterative nature limits GPU parallelization. Consequently, GJK is less suited for dynamic SPH simulations, where particles frequently interact. [8]

A Bounding Sphere offers fast and efficient collision detection, particularly for spherical objects. Its rotation invariance and simple distance-based checks make it highly parallelizable. However, bounding spheres poorly approximate irregular or elongated shapes, leading to false positives. Additionally, high-speed interactions may require supplemental detection methods. [9]

The Separating Axis Theorem (SAT) determines collisions by identifying an axis along which objects can be separated. Effective for convex polygons and polyhedra, SAT struggles in SPH fluid simulations due to the high computational cost of testing multiple axes. Its inherently sequential nature further limits GPU and multi-core parallelization, reducing efficiency in densely packed particle environments. [10]

In summary, broad-phase algorithms like BVH, uniform spatial subdivision, and quad-trees provide effective early-stage collision filtering, while narrow-phase algorithms such as GJK, bounding spheres, and SAT refine detection accuracy. However, trade-offs exist between precision, computational efficiency, and parallelization potential, impacting their suitability for real-time SPH simulations.

Broad and Narrow Phase Algorithms

It was decided to use uniform spatial subdivision as the broad-phase algorithm in our design. Uniform spatial subdivision is less computationally intensive than BVH or quad-trees, which will improve performance, especially in a real-time simulation.

The primary reason for not implementing a binary tree-based lookup instead of a spatial hash was due to complexity and performance. For similarly-sized particles, which is a large majority of our fluid simulation (even with variance of particle sizes), spatial hash lookup and maintenance is much faster than any competitors, as both are $O(1)$ with a good hashing algorithm and only $O(n)$ complexity to fully generate the initial hash for all particles. Aside from being harder to implement, tree-based lookups such as quad and oct-trees have $O(n(\log n))$ complexity for generation and much slower lookup and maintenance speeds, requiring either a search through the tree or rebalancing; of which both have $O(\log n)$ complexity [11]. As such, performance at scale would generally be much worse using tree-based algorithms for our use case, as our particles are generally similarly-sized.

As a result of our analysis of the narrow-phase algorithms, it was decided to use bounding sphere narrow-phase collision detection as it offers the best performance for real-time applications involving spherical particles, such as SPH. It is also very parallelizable, which will allow for high performance as well as opportunities to split the workload across the CPU and GPU.

Choice of Sorting Algorithms

Purpose

In broad-phase collision detection, efficient spatial subdivision is critical for optimizing object-to-object interactions. Sorting algorithms are used to organize particles which have been assigned grid cell IDs. Sorting aligns them in memory space, enabling fast neighbor searches and reducing the computational complexity of physics kernels. It is a key step in our collision detection algorithm. As such, choosing a good algorithm that runs excellently on our GPU improves our overall performance substantially, while also improving the proportional performance of our heterogeneous algorithm by reducing the amount of processor time spent on non-heterogeneous tasks.

Bitonic Sort

Our initial implementation used *bitonic sort* for uniform spatial subdivision. Bitonic sort operates by recursively comparing and swapping pairs of bitonic sequences, merging them at each stage to produce a sorted dataset. While bitonic sort is inherently parallelizable, it suffers from poor scalability due to its high time complexity ($O(n \log^2 n)$) and significant overhead from data dependencies. These dependencies arise from the need to repeatedly merge sorted sequences, which limits performance as the number of elements (n) and the degree of parallelism increase. As a result, bitonic sort became a bottleneck in our simulation, particularly for large-scale particle systems, and hindered the overall efficiency of our simulation [13].

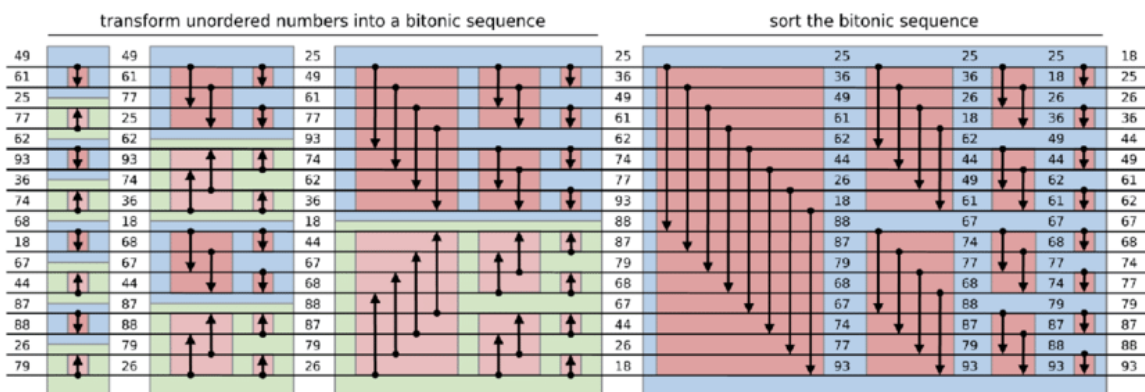


Figure 1: Visual depiction of bitonic sort [12].

Radix Sort

We also explored *radix sort* as a potential alternative. Radix sort is a non-comparative sorting algorithm that processes elements digit-by-digit or byte-by-byte, grouping them into buckets based on the value of the current digit. This approach has a time complexity of $O(nk)$, where k is the number of digits or bytes in the key. Radix sort is highly parallelizable and exhibits good spatial locality, making it suitable for GPU implementations. However, its performance is highly dependent on the choice of radix and the distribution of input data. While radix sort outperforms bitonic sort in many cases, it still introduces overhead from multiple passes over the data and requires careful tuning to achieve optimal performance [14]. In the context of our

heterogeneous algorithm, radix sort showed promise, however its algorithmic complexity appeared generally worse than our final choice, *counting sort*.

Counting Sort

Ultimately, we adopted *counting sort* for our uniform spatial subdivision algorithm. Counting sort works by computing the position or rank of each element using a count array (in this case, the count of particles in each grid cell). Final element positions are determined by summing the counts, which can be efficiently computed using a parallel prefix sum. Counting sort has a time complexity of $O(n + k)$, where k is the size of the count array, and it scales exceptionally well in parallel contexts due to its low memory overhead and minimal data dependencies [15]. By leveraging GPU-accelerated prefix sums, we achieved a significant performance improvement of 10-20% compared to bitonic sort, depending on the system configuration. This improvement directly enhanced the efficiency of our simulation by reducing the GPU's sorting time and allowing more resources to be allocated to other tasks.

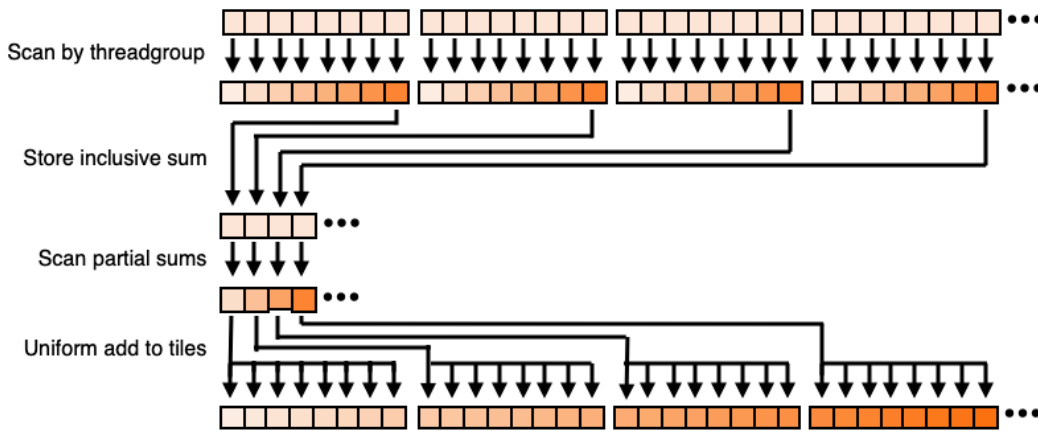


Figure 2: Visual depiction of counting sort [16].

Our Final Choice

After evaluating the trade-offs between bitonic sort, radix sort, and counting sort, we selected *counting sort* as our final choice for uniform spatial subdivision. Counting sort's superior scalability, low memory overhead, and efficient use of GPU parallelism made it the best fit for our simulation, leading to an improvement in average frame rates compared to bitonic sort. By optimizing this GPU-only portion of our heterogeneous algorithm, we were able to significantly reduce the time spent on non-heterogeneous tasks, which improved our simulations overall performance, and should also improve the proportional performance of any heterogeneous algorithm we implement.

Evaluation Metrics and Testing Methodology

In real-time applications such as games, performance is primarily measured using FPS (frames per second) or frame time [17]. For this project, frame time was chosen as the primary evaluation metric, with the objective of reducing it compared to a baseline measurement.

Additionally, wavefront (or warp) completion time was selected as a secondary metric. Given that the project aims to distribute the workload across both the CPU and GPU, using a combination of frame time and wavefront completion time provides a comprehensive evaluation of the workload distribution's effectiveness. Specifically, the heterogeneous CPU-GPU collision detection system should demonstrate a reduction in overall frame time compared to a GPU-only baseline.

To evaluate simulation performance, a structured testing methodology was developed to compare the CPU-GPU collision detection implementation with a GPU-only baseline. The testing process involved two programs: one designed to monitor overall performance over time and another that performed a detailed single-frame analysis using external profiling tools. The overall performance was assessed by measuring the average CPU and GPU frame times over a 10-second duration, using **at least** three different particle counts (e.g., 1,000, 16,000, and 100,000) in both single-fluid and multi-fluid simulations to generate multiple data points for performance evaluation and bottleneck identification. The first program, integrated into the simulation, gathered this frame-time data. Meanwhile, single-frame performance was analyzed using third-party GPU profiling tools (such as Radeon Developer Suite or Nvidia NSight) to gain insight into wavefront execution. An example simulation used for testing is illustrated in the figure below.

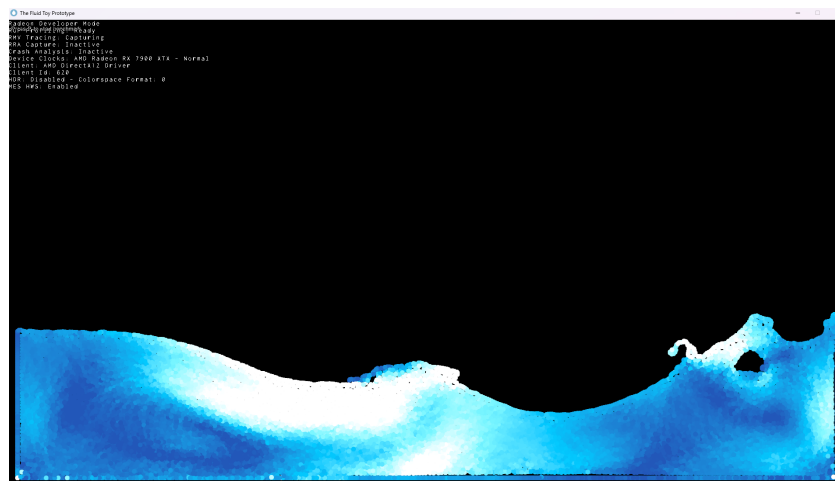


Figure 3: Particle Simulation Testbench.

GPU-Only Prototype for SPH Simulation

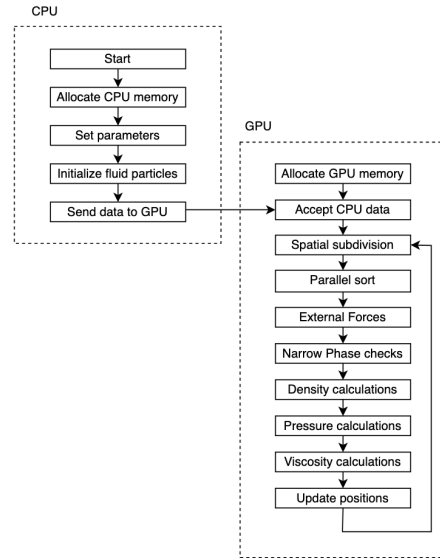


Figure 4: High-level flowchart of GPU-only implementation.

The GPU-only prototype serves as the baseline for performance evaluation and optimization efforts, providing a reference point before incorporating workload-balancing improvements. This implementation, built using Unity’s compute shaders, exclusively runs on the GPU without heterogeneous processing. The system employs Smoothed Particle Hydrodynamics (SPH) and focuses on efficient collision detection and fluid simulation.

Design and Implementation

The simulation utilizes a particle system where each particle is represented as a structured array containing position, velocity, and density data. Particles are initially arranged in a grid, and compute buffers manage data storage and manipulation for parallel processing. The core simulation loop consists of multiple iterations per frame to enhance stability. Key computational kernels include:

- **External Forces** – Applies gravity and user interactions.
- **Update Spatial Hash** – Uses a hashing function to partition space into a grid for efficient neighbor searches.
- **Sort and Calculate Offsets** – Organizes particles by subdivision cells using the parallel counting sort algorithm.
- **Calculate Densities & Pressure Forces** – Computes fluid densities and applies pressure forces via smoothing kernels (e.g., Poly6, Spiky).
- **Calculate Viscosity or friction** – Simulates viscosity by aligning particle velocities with their neighbors.
- **Calculate Friction** - Calculates frictional forces between solid particles (referred to as *powders*), which generates a force vector based on the frictional coefficient of the powder and the force of its normal vector, relative to its velocity

- **Calculate Temperature** - Mimics heat transfer through the use of a thermal diffusivity constant tied to the particle to equalize particle temperatures with surrounding particles
- **Update Positions & Collisions** – Finalizes particle positions and resolves collisions. The system also includes interactive elements such as mouse-based force application, pause/resume controls, and adjustable parameters for gravity, pressure, viscosity, friction, and temperature.

Rendering and Visualization

Particles are rendered using GPU instancing for performance efficiency, with each particle displayed as an instanced mesh. A gradient-based shader determines color based on velocity, while alpha blending creates smooth, visually appealing particle effects.

By altering the physical and visual properties of each particle, we were able to simulate different types of fluids dynamically at runtime. However, to maintain memory coherence and spatial locality, we could not store the properties of each particle within the particle struct itself, as doing so would have led to fewer cache hits (since the particle structures would have been spaced further apart in memory) and increased memory usage due to duplicated data. To prevent this issue, we adopted a data-oriented approach. First, we created a general fluid class that each fluid type implemented. Then, at start-time, we sent the compute shader (responsible for physics calculations) and the visual shader (responsible for drawing each particle to the screen) a table of all the properties needed for every possible particle fluid type in the scene.

The compute shader received an array of structs, with each struct containing the physical properties of a single fluid type. Similarly, the visual shader received an array of structs containing visual properties for each fluid type, though it required some preprocessing of this data. Each particle contained a one-dimensional gradient texture (64px by 1px in resolution) used for visual effects, such as changing the particle's color based on its speed. We stitched together this data from all particles to form a single texture, making it easier for the visual shader to process. For example, in a scene with seven fluid types, we created a 64x7 resolution 2D texture.

For both the physical and visual shaders, each thread referenced the necessary data from the tables by checking a single integer flag stored within each particle's struct. There was one thread per particle, and each wavefront consisted of 32 to 64 threads. These threads loaded the entire required table into memory to speed up data access. Figures 5-10 showcase rendering of various agitated fluids within the simulation environment.

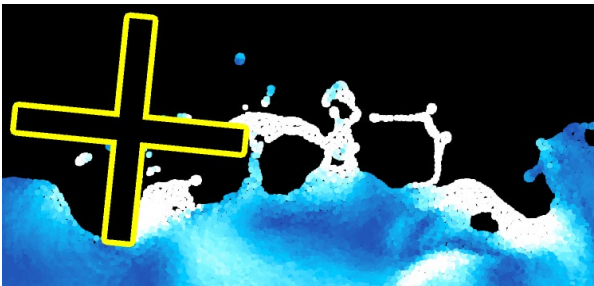


Figure 5: Demonstration of Propellor Agitating Water Fluid Type



Figure 6: Demonstration of Propellor Agitating Honey Fluid Type

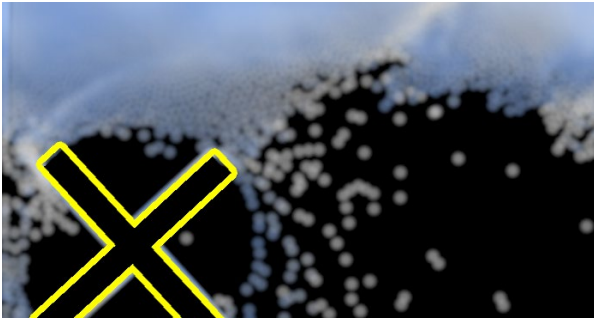


Figure 7: Demonstration of Propellor Agitating Steam Fluid Type

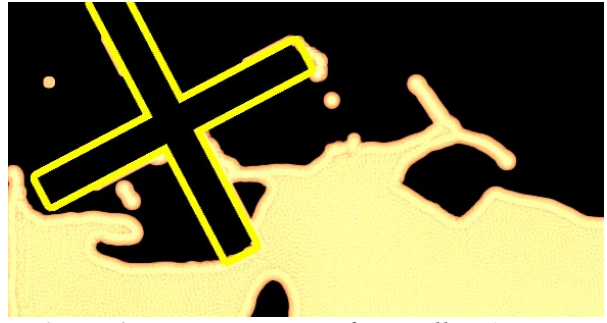


Figure 8: Demonstration of Propellor Agitating Lava Fluid Type

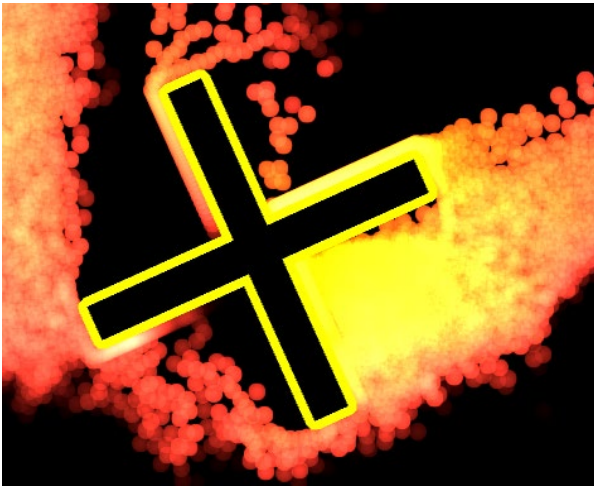


Figure 9: Demonstration of Propellor Agitating Fire Fluid Type



Figure 10: Demonstration of Propellor Agitating Snow Powder Type

Performance Analysis of GPU-Only Prototype

To generate a baseline GPU-only measurement a wavefront analysis was performed for a simulation of multiple fluids with varying particle sizes, with a total particle count of 307200. The figure below shows the three largest kernel dispatches within the simulation, specifically density, pressure, viscosity, and temperature.

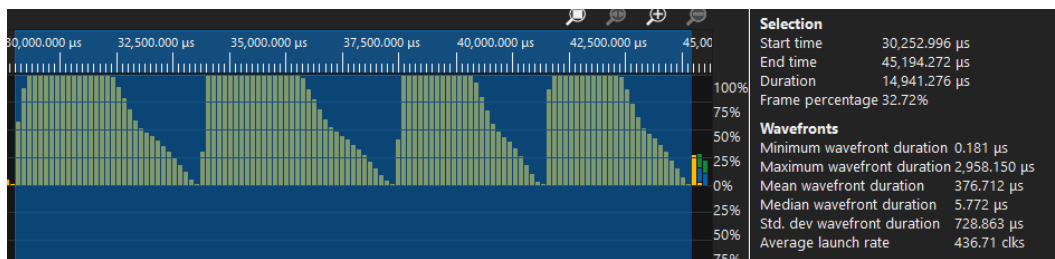


Figure 11: GPU utilization for density, pressure, viscosity, and temperature dispatches.

Analysis of the wavefronts revealed a significant difference between the maximum and minimum wavefront durations, and a very large standard deviation of wavefront durations. The high variability in wavefront durations indicated poor wavefront coherency which was limiting performance significantly. A theoretical analysis was performed to determine the cause of the poor coherency.

Based on the text *Real-time Collision Detection* by C. Ericson [18], the hypothesis of the cause of poor wavefront coherency was poor broad phase collision detection. The uniform spatial subdivision grid was too coarse, leading to a large increase in computationally expensive narrow phase checks and physics calculations that limit performance [18]. This hypothesis was based on how uniform spatial subdivision is optimized for collision detection. Uniform spatial subdivision requires an optimal subdivision cell size to properly predict possible collisions [18]. For the simulation, the cell size must be set to be the same as the size of the largest particle in the simulation to avoid missed collisions. However, since the cell size of the simulation depends on the largest particle, in simulations with multiple fluids, broad phase collision detection for smaller particles becomes significantly worse, which leads to a significant increase in narrow phase collision checks and physics calculations that would otherwise not be performed if the cell size was set optimally for said smaller particle size.

This performance bottleneck was the impetus for designing an optimization technique called particle-based subdivision, with the goal of eliminating the bottleneck through heterogeneous CPU-GPU computing.

Particle-Based Subdivision Design

To eliminate the uniform spatial subdivision cell size bottleneck seen in the GPU-only implementation, we designed an approach called *Particle-based Subdivision*, which subdivides fluid types based on the fluid's particle size. By subdividing the particles into two groups based on their size, the CPU then performs collision detection for the smallest group, while the GPU performs collision detection for the largest group.

This design focused on maintaining high coherency on the GPU by ensuring that all particles on the GPU are of a similar size. This avoids the mismatch between particle size and subdivision cell size that results in increased narrow phase calculations but also introduces periodic data transfers between the CPU and GPU. However, by avoiding extra physics computations on the GPU it is hypothesized that the increase in frame time due to the data transfer between the CPU and GPU is less than the frame time reduction from the more optimal broad phase collision detection on the GPU. A flowchart demonstrating this heterogeneous split can be seen in **Figure 12**.

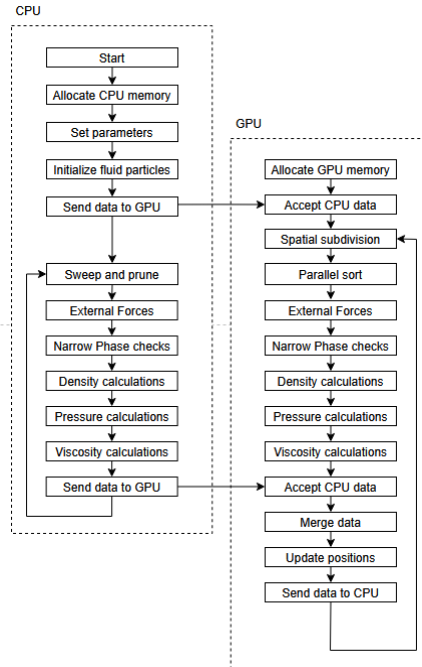


Figure 12. Particle-Based Subdivision Flowchart

The particle-based subdivision algorithm allows objects (particles or non-particle colliders like boxes) to be simulated on the CPU while the GPU performs the more significant and computationally expensive tasks such as fluid simulations and rendering. By design, the number of particles assigned to the CPU will be smaller than the number of particles assigned to the GPU, allowing the CPU to spend extra cycles calculating physics interactions of more complex non-particle colliders, as well as executing game logic and processing inputs.

The performance of particle based subdivision is expected to be tied to the difference between the smallest and largest particle sizes within a simulation. Simulations with very similar particle sizes should perform worse or equal when running on CPU and GPU compared to a GPU-only baseline. This is expected since particles of similar size have similarly sized optimal subdivision cell sizes and so would not have any significant performance impact. Conversely, simulations with significantly different minimum and maximum particle sizes are expected to have a massive increase in performance compared to a GPU-only baseline.

Particle Structure and Initialization

Each particle in the simulation was modeled as a struct with 6 attributes:

- Near density and density - float representations (8 bytes total)
- Velocity - 2D vector representation (8 bytes total)
- Predicted position - 2D vector representation (8 bytes total)
- Position - 2D vector representation (8 bytes total)
- Temperature - float representation (4 bytes)
- FluidType - enumerated integer representation (4 bytes)

During simulation, the attributes of each particle are updated as the particle moves within the simulation space. At initialization, the CPU creates an array of particle structures of a length predetermined by the developer. Each particle in the array has its position updated such that the particles are all evenly distributed within the spawn region. The FluidType attribute of each particle is also updated based on the predetermined selection by the developer. Once the array of structs is created, the CPU transfers all particle data to the GPU.

To reduce the size of each particle structure, a new structure was designed which holds all physics/calculation constants for a specific fluid type. The developer can specify which fluids will be in the simulation, or if none are specified all fluids will be loaded to the GPU. The fluid type with the largest particle size that is in the simulation will dictate the subdivision cell size, so choosing the fluids ahead of time is better for performance.

SPH Simulation Parameters

Simulation Hyperparameters

The fluid simulation features a set of hyperparameters primarily influencing the performance of the overall simulation. Hyperparameters include simulation time scale, fixed time step enable, max particle count, iterations per frame, global entropy rate, room temperature, and max spawn rate. Hyperparameters are adjustable on a per-stage basis in the Unity editor.

Simulation time scale, fixed time step, and iterations per frame all influence the chronology of the simulation. Adjusting the time scale will affect the speed at which time is perceived to pass in the simulation, and adjusting the iterations per frame will increase the number of simulation steps calculated each frame which increases the dispatch calls and stability of the simulation. Setting a fixed time step disables the dynamic time step optimization in the simulation, which fixes the size of the time step to a hard-coded constant (1/120 or 0.0083 at time of writing). The primary purpose of having fixed time steps is to improve overall simulation stability as well as maintain fixed behavior across a wider band of frame times. Dynamic time step scaling can sometimes lead to undesired or variable simulation behaviour at the extremes.

Max particle count and max spawn rate are both self-explanatory. The former sets a hard cap on the number of particles that may be present at once in a scene, while the latter caps the number of particles that may be spawned in a single simulation step. A max particle count is required due to the spatial subdivision and counting sort algorithms used in our implementation, which both require knowledge of the maximum number of grid cells or particles ahead of time to function. It additionally serves as a control to ensure a baseline for simulation performance, as too many particles may crash the application systems with limited resources. Max spawn rate acts as a way to synchronously add a controlled amount of particles to the scene instead of having every GPU thread immediately spawn in the particles.

Finally, global entropy rate and room temperature both directly affect the specific scene in the simulation. Room temperature sets a constant value in the scene as a target temperature for particles to approach, which simulates loss of thermal energy to the universe through entropy, with global entropy rate linearly scaling the rate at which this energy loss occurs.

Fluid Parameters

Different fluids in the simulation are separated by their fluid parameters, essentially a set of values that determine constants for particles to take on in the simulation. Fluids are identified with an enumerator, *FluidType*, which is used to map particles to sets of fluid constants, called *FluidParams*. In simulation, individual particles only need to store a reference to a *FluidType* ID, which is then mapped at run-time to a table of available fluids in order to determine particle behaviour for each physics kernel. This substantially reduces memory use for many particles, and enables fluids to be modular dynamically loaded for each scene. The superset of a *FluidParam* is *FluidData*, which also contains *VisualParams*, additional shader information for use by Unity to render the visual appearance, separated to reduce the amount of data to transfer when dispatching kernels for fluid simulation. Fluids are stored as ‘scriptable object’ Unity assets, with values that may be adjusted in the editor to tweak target density values, thermal conductivity, temperatures for state change, smoothing radii, visual appearance, etc..

CPU-GPU Data Transfer Design

After each simulation frame, the CPU transfers the positional data and size data of each CPU object to the GPU. This data transfer allows CPU objects to interact with the fluid simulation on the GPU, improving simulation quality. This data transfer is expected to increase frame time but it is expected that the reduced narrow phase checks and physics calculations from the particle-based subdivision will result in a more significant reduction.

In the event that the CPU needs to read back the particle data from the GPU (e.g., for game logic), the CPU makes an asynchronous read request to the GPU. The asynchronous read request is a non-blocking read request that allows the CPU to continue performing useful work while the readback is performed in the background. Asynchronous read requests are implemented in Unity using Unity Engine’s *AsyncGPUReadback* class. The readback request functions as an interrupt that can be configured to call a specific interrupt handler. This allows for non-blocking read requests from the GPU which improve performance and allows for WebGPU compatibility.

The two primary use cases for asynchronous readbacks in The Fluid Toy come from density detectors and thermal sensors. Density detectors calculate the particle density within its specified radius, effectively allowing for the detection of fluids and evaluating the number of particles in a space. The thermal sensor evaluates the average temperature within its specified radius, which was a useful feature for level design. Both sensor types use a manager class that makes a readback request and then handles the sensing for all connected sensors once the read request is completed.

Feature Implementations

Dynamic CPU-GPU Device Partitioning

CPU-GPU particle partitioning can be enabled and automated on a per-scene/level basis. Solid collider objects are automatically processed on the CPU side, with particles defaulting to the

GPU. Particle-based subdivision can also be automated to process the simulation for specific particles on the CPU as necessary.

Unified Physics Kernel

In an effort to increase performance, the kernels for pressure, viscosity, friction, and temperature were combined into a single kernel. This has the effect of reducing the number of kernels dispatched onto the GPU, and makes management of the kernels simpler. It also reduces the total amount of memory reads needed for the broad-phase nearest neighbour checking algorithm.

Platform Specific Simulation Resolution Scaling

Platform-specific resolution scaling is implemented as a linear scaling algorithm that modifies the volumetric size of particles with inverse proportionality to the number of particles, preserving overall fluid volumes while reducing total number of particles. The primary purpose being to improve simulation performance, particularly when targeting mobile web platforms.

Level Designs

To showcase our simulation's capabilities and features in a real time gaming application we plan to develop a series of mini-game levels. Each of which will increase in difficulty and complexity in order to ease the player into the gameplay and teach them how to interact with the simulation. Each level should feature a different aspect of the simulation and will have various themes and control schemes. Each level will also have a scoring system in order to increase replayability and to add a fun competitive nature. The score for each level can be calculated based on a simple timer but it is implemented in a way that it could feasibly be calculated based on other unique metrics per level. During development it may be necessary to split up some level ideas between multiple sub-levels.

Level 1: Tavern Training

Input Method: Left and Right click with the mouse to use the gravity brush.

Theme and Goal: The fluid is themed to look like beer and the level has a tavern aesthetic. We will agitate the beer to make the level more chaotic and fun and the user will be timed on how fast they can fill the cup with beer. Once the cup is filled, a fluid detector will trigger the level completion animations which will indicate that the level is done and it will display how well the player did. The high score of the player will then be saved to the disk and displayed on the level select screen.

Implementation Details: The cup's movements are simulated on the CPU. Its position, size and shape is then sent to the GPU where all the fluid particles are also simulated. Collisions between the fluid and the cup are calculated on the GPU and the fluid particle movements are updated in parallel on the GPU. Then the GPU sends back the positions of the fluid particles to the CPU which checks if the cup has been filled with fluid yet to end the level.



Figure 13: Level 1 Implementation

Level 2: Carnival Chaos

Input Method: Aim water gun at bottom of screen with mouse. Left click to shoot

Theme and Goal: This level has a carnival theme, the goal of the level is to have the player aim a wobbly water gun to shoot a target and fill up a water meter. There are rows of moving obstacles that occasionally block the line of sight for the player and the target is moving as well. The player is scored based on time to complete level and water wasted.

Implementation Details: The movement of the target, obstacles, and water gun are simulated on the CPU. When the player presses the left mouse button, the water gun sprays water in the direction of the player's mouse. A blue meter on the right indicates how much water is left in the water gun before it needs to be recharged, and the meter on the left indicates how many more times the player needs to hit the target to win the level.

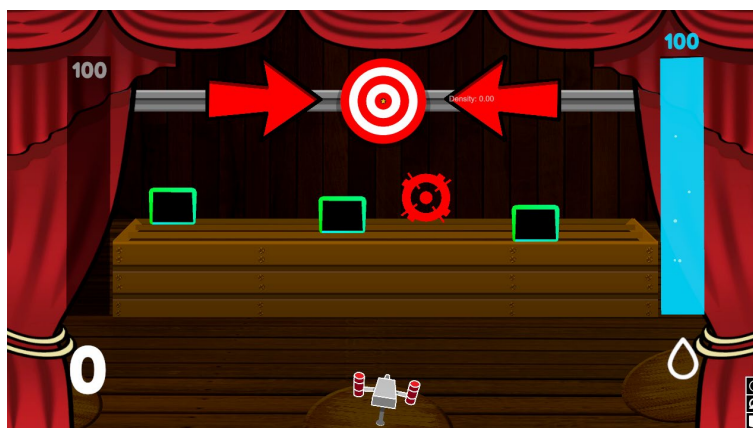


Figure 14: Level 2 Implementation

Level 3: Lava Laboratory

Input Method: Left and Right click with the mouse to use the gravity brush.

Theme and Goal: This level has a laboratory theme; the goal of the level is to have the player mix different temperature lava together to get the target temperature. The player needs to use the gravity

brush to mix the lava fluids together to get the right temperature. The player is scored based on time to complete level.

Implementation Details: The cup movements are simulated on the CPU. Each cup is filled with lava fluid particles at different temperatures: 1200, 1000, and 700 Celsius. The player uses the gravity tool to pick up the lava fluids and place them into the empty cup. The player must try to get the temperature of lava in the cup to 900 degrees Celsius, with a 1% tolerance.



Figure 15: Level 3 Implementation

Level 4: Sliding Door Puzzle

Input Method: Use the mouse to slide open a close doors

Theme and Goal: There will be multiple chambers and tunnels containing water. The goal of the level is to open the doors in the right sequence such that you can find and press the victory button. The player will be timed based on how fast they complete the level.

Implementation Details: There are two interactable “gates” that the player can interact with, colored green. To complete the level, the player must first pour water onto the electrical panel (pictured top right), which will cause a fire.

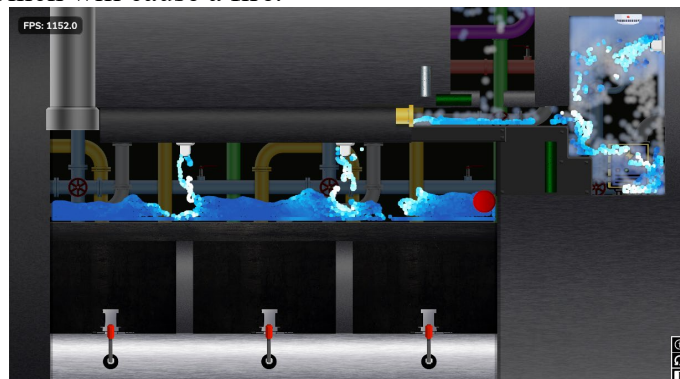


Figure 16: Level 4 - Water on the electrical panel causes a fire.

The fire from the electrical panel sets off the smoke detector (pictured in top right) and the sprinkler system activates, putting out the fire but also flooding the boiler chamber with water (pictured in the middle). The victory button can now be seen by the player as a red button in the

bottom right corner of the boiler chamber. However, clicking the button will not complete the level until the water in the boiler chamber is removed.

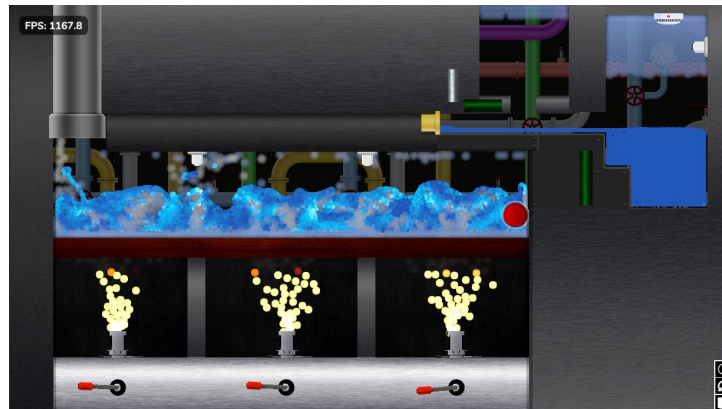


Figure 17: Level 4 - Burners causing water in the boiler chamber to turn to steam.

To boil away the water the player must precisely adjust the fuel valves (pictured at the bottom below the burners) until the bottom of the boiler chamber starts to heat up and the water begins to boil away. As the water boils away the victory button begins to turn green, indicating to the player that their actions are correct. Once the water and steam is sufficiently removed from the chamber, the button unlocks and the player pressed the button to win the level.

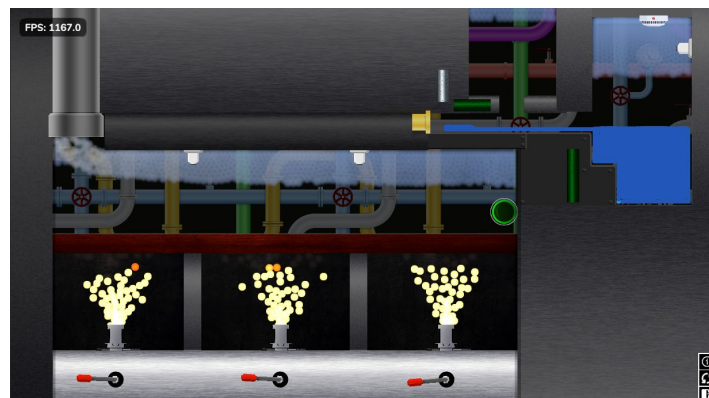


Figure 18: Level 4 - Water boiled to steam and vented out of the boiler chamber

Level 5: Planet Annihilation

Input Method: Use the mouse to select and aim different weapons at the planet.

Theme and Goal: This level will have a space theme. The goal of the level is to completely destroy the planet using the four weapon types provided. The player selects a weapon and aims it at the planet to damage and remove the planet. The player is scored based on how long they take to destroy the planet.

Implementation Details: The planet is created by immediately spawning in elements such as stone, lava, and snow at the start of the level. The radial gravity system which points all gravity towards

the center of the screen, coalesces these particles into a circle resembling a planet. Then, the player uses source objects to emit high velocity laser beam particles to destroy the planet.

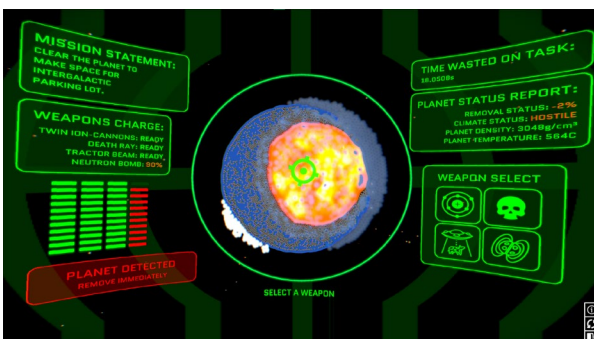


Figure 19: Level 5 - Image of the planet to be destroyed



Figure 20: Level 5 - Image of the planet being destroyed by laser beam

Level 6: Casting a Sword

Input Method: Use the mouse to move the cast and open the crucible.

Theme and Goal: This level will have a medieval/blacksmith theme. The player needs to align the sword cast with the crucible, pour the liquid metal into the crucible and wait for it to solidify. Once solidified the sword is dropped into a cooling basin with water to cool. The player is scored based on the quality of the casting.

Implementation Details: First the metal is poured into the cast by the player. The player must take care not to spill any metal as that would result in a poor grade for the finished sword. Once all the metal is in the cast, the player needs to wait for the sword to cool. The player should avoid bumping or moving the cast as the sword is cooling as this can result in a poor quality result. Once the sword has solidified, a lever will appear that will open the cast and drop the sword into a water basing to cool it. Finally the sword is removed from the basin once cooled and the player wins the level. The player's star count depends on the grade of the casting.

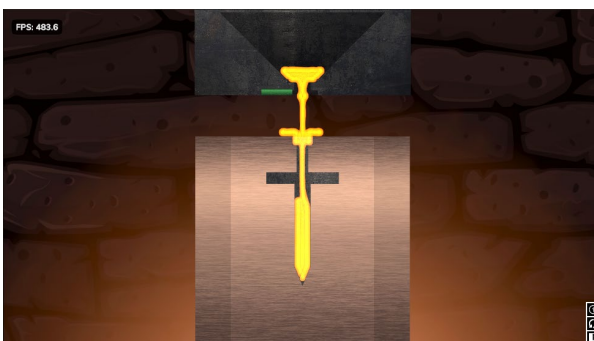


Figure 21: Level 6 - The cast is moved into position and the molten metal is poured into it.

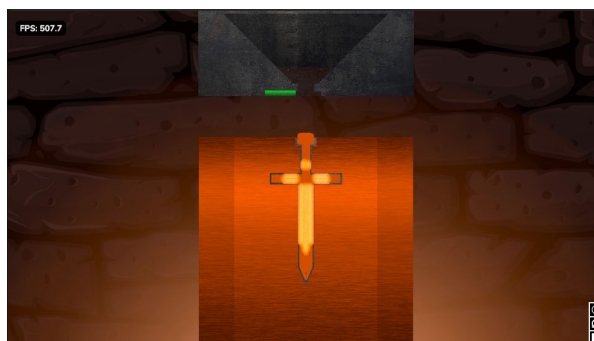


Figure 22: Level 6 - The sword cools and solidifies within the cast.



Figure 23: Level 6 - Sword cooling in water basin

Level 7: Drink Delivery

Input Method: Mouse to move around containers.

Theme and Goal: The inspiration for this level will be fast-food drink delivery. The player will have to rapidly fill and deliver cups of various drinks in a fixed order as fast as they can. Scoring will be based on how full the cups are, as well as how quickly orders are delivered.

Implementation Details: The level interface presents the users with a list of drink orders (drink type and size). One set of buttons are present for generating small, medium, and large cups; while another set of buttons dispense the corresponding drink.

Users will need to generate the correct cup, fill it to almost-full, and carefully drag the cups into an “order zone” in order to score. The challenge of the level is to do all of the above both quickly, and without spilling the drink contents in the process (or else they may lose out on scoring!).

Levels In Development:

The remaining levels in development explore diverse gameplay mechanics and themes. "Drink Delivery" challenges players with rapid-paced fast-food service, requiring them to fill and deliver drink orders accurately and quickly without spillage. "Plumbing Pipes" presents a puzzle where players manipulate pipe fittings to direct water flow to a destination, incorporating obstacles and pressure control. "FireFighter Training" introduces character control (WASD) as players navigate a burning building to extinguish fires, adding a health and time-based scoring system. "Humble EarthWorm" features a survival challenge where players control a worm using WASD to traverse a map and avoid deadly downpours, with scoring based on remaining health and completion time. "Chemistry Set" focuses on interactive chemistry tasks, such as distillation, scored on completion speed. "Planet Builder" offers a creative experience where players draw and populate a planet with various elements. Finally, "Waterbender" introduces character movement (WASD) and mouse-controlled telekinetic water powers to combat fire and lava enemies, with scoring based on remaining health and completion time.

Sandbox Level

This level which is accessed directly through the main menu was designed as a free play space where the player can use and experiment with all 40 of the fluids and objects that we have developed for the simulation. It also serves as a benchmark level that we will use to evaluate the performance of the fluid simulation. It works well as a learning tool for concepts such as water erosion, surface tension, glacial melt and hydrostatic pressure. This level will be updated to include more features as we develop the game, allowing us to test all features in a single environment, streamlining the testing process.

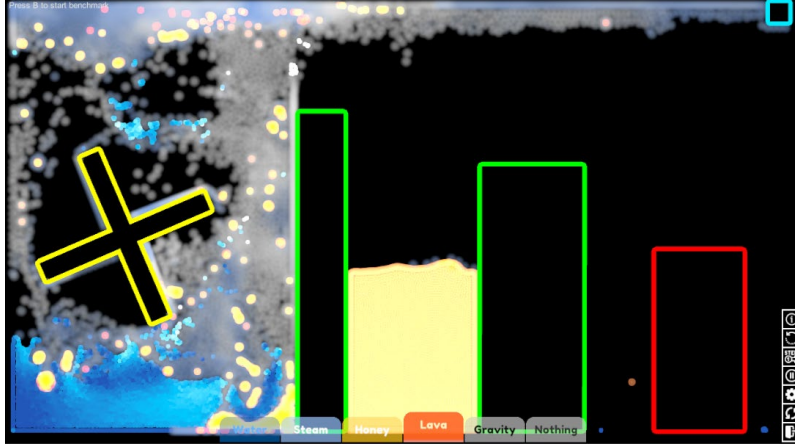


Figure 24: Showcase of Sandbox scene, with multiple fluids, and multiple box colliders.

Alternative Designs

Density-based Subdivision Design

An alternative design that was developed alongside *particle-based subdivision* was *density-based subdivision*, where the subdivided grids of particles stemming from our broad-phase collision detection algorithm are categorized by particle density, with high-density, highly-sequential grid kernels dispatched to the CPU. This was the original approach taken to the workload split, however performance was consistently worse than the GPU-only approach, particularly when the algorithms and memory management of the original simulation was improved. As a result, this approach has since been retired, but will be briefly described in this section for sake of completeness.

Implementation of Density-based Subdivision

There are two methods we used to achieve a density-based workload split, hereby referred to as *method 1* and *method 2*:

1. Obtain particle counts for each cell during cell sorting, before any kernel calculations and dispatch the cells with highest and/or lowest population to the CPU instead of the GPU.
2. Obtain particle counts for each cell from the previous frame, running the first frame of the simulation in GPU-only, and dispatch the previous highest and/or lowest density(population) to the CPU for subsequent iterations.

Method 1 attempts to improve performance proactively by leveraging the heuristic that cells with the highest populations have highly sequential computations and therefore are most likely to be the largest contributor to the increased frame time. To implement this heuristic, The sorting algorithm, which sorts the particles based on their subdivision cell, is modified to also calculate and store the top N (variable) highest population cells and a data transfer is made to send the index of these cells as well as the particle data to the CPU. The SPH calculations for those cells are then performed on the CPU and the resulting particle data is transferred back to the GPU and merged with the GPU particle data.

Method 2 attempts to improve performance by taking a retroactive approach while leveraging the same heuristic. Using data from a previous calculation, the retroactive approach moves the GPU to CPU data transfer of position and velocity data to the end of the first iteration. For subsequent iterations, the CPU computes the density, pressure, viscosity, and position data for the top N highest density locations and then sends the data back to the GPU to be merged.

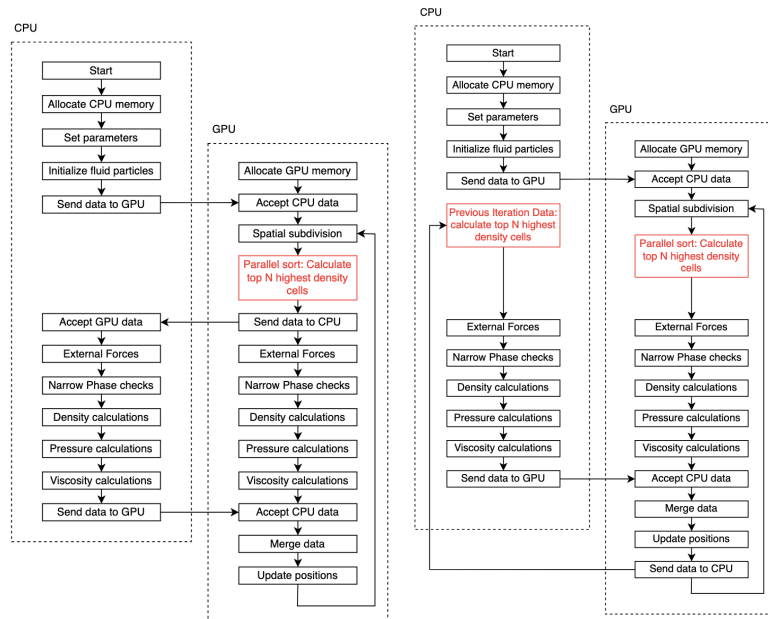


Figure 25: Comparison of workload split 1 (left) and 2 (right).

Left side (1) obtains approximate particle counts during sort and sends position and velocity data before computation, right side (2) returns the position and velocity data after the first iteration. To implement method 1 the SPH fluid calculation kernels are first ported to C# to add SPH calculation functionality to the CPU. The sorting kernel on the GPU is modified to calculate a *leaderboard* of the highest population cells using a new structured buffer. This *leaderboard buffer*, as well as the sorted position data and velocity data is then sent to the CPU. The CPU then computes the new positions and velocities for the top N highest population cells in parallel with the GPU. The GPU calculation kernels are modified to not compute the cells that were sent to the CPU. The results from the CPU are then passed back to the GPU for rendering. This method has the data transfer to the CPU preceding the calculation stage on the GPU and this data transfer is a blocking process, which may cause a performance decrease if the data transfer time is too long.

The implementation of method 2 also requires the calculation kernels to be ported to C#, but is simpler, only requiring the data transfer from GPU to CPU to occur after the first iteration of the SPH algorithm has completed. During the next iteration, the CPU uses the density data to calculate the top N highest density cells, and then computes the SPH position and velocity data. The GPU compute kernels are modified as in method 1 to compute the top N highest population cells and not compute those cells. This method moves the data transfer from GPU to CPU to the end of the calculations stage and does not create a blocking data transfer between the sorting kernel and the SPH calculation kernels. Method 2 reduces the number of data transfers required down to only 1. However, method 2 introduces a potential data parity hazard as it requires the CPU and GPU to always calculate the same top N highest population cells, otherwise certain particles will not be calculated on the CPU or GPU.

Initial performance benchmarks were below targeted improvements of 10%, and the approach had to be scrapped as it did not appear to be feasible. Full performance of density-based subdivision can be seen in the *Performance Measurements* and *Analysis of Performance* sections.

Measurements and Procedure

To evaluate the performance of the GPU-only and Heterogeneous CPU-GPU fluids simulations, the testing methodology focused on evaluating the performance both in single-fluid simulations and multi-fluid simulations. There were ten total scenarios tested, the full table of scenarios can be found in **Table 1**. The performance improvement from the CPU-GPU implementation was hypothesized to be linked to the difference between the largest and smallest particle sizes for the fluids being simulated as well as the total number of particles in the simulation. From this hypothesis the test scenarios were targeted specifically at evaluating performance deltas with respect to the difference between largest and smallest particle size, as well as the number of particles in simulation. All tests were performed on the same system hardware (Ryzen 7700x CPU, and Radeon 7900XTX GPU running Windows 11).

Three fluid types were created for testing: *water*, *small bouncy ball*, and *large bouncy ball*, with particle sizes (smoothing radius) 0.35, 1, and 2 respectively. Each test scenario is executed five times and the average frame time is calculated. Each test scenario is also executed at five different total particle counts, specifically 1024, 10240, 102400, 204800, 307200. Fixed-time step is a feature that can significantly impact performance and so testing with fixed-time step enabled and disabled was performed to ensure the accuracy of performance results.

Table 1. GPU-only vs Heterogeneous CPU-GPU Fluid Simulation Test Scenarios.

Test Scenario #	Test Name:	Fixed-Time Step (FTS):	Fluid Types in GPU Simulation:	CPU Object Types in Simulation:
1	GPU-only, Water-only FTS Disabled	Disabled	Water	None
2	GPU-only, Water-only FTS Enabled	Enabled	Water	None
3	GPU Water & Small Bouncy Ball FTS Disabled	Disabled	Water, Small Bouncy Ball	None
4	GPU Water And Small Bouncy Ball FTS Enabled	Enabled	Water, Small Bouncy Ball	None
5	GPU Water And Large Bouncy Ball FTS Disabled	Disabled	Water, Large Bouncy Ball	None
6	GPU Water And Large Bouncy Ball FTS Enabled	Enabled	Water, Large Bouncy Ball	None
7	GPU Water CPU Large Bouncy Ball FTS Disabled	Disabled	Water	Large Bouncy Ball
8	GPU Water CPU Large Bouncy Ball FTS Enabled	Enabled	Water	Large Bouncy Ball
9	CPU-GPU Water Split FTS Disabled	Disabled	Water	Water
10	CPU-GPU Water Split FTS Enabled	Enabled	Water	Water

Performance Measurements and Analysis

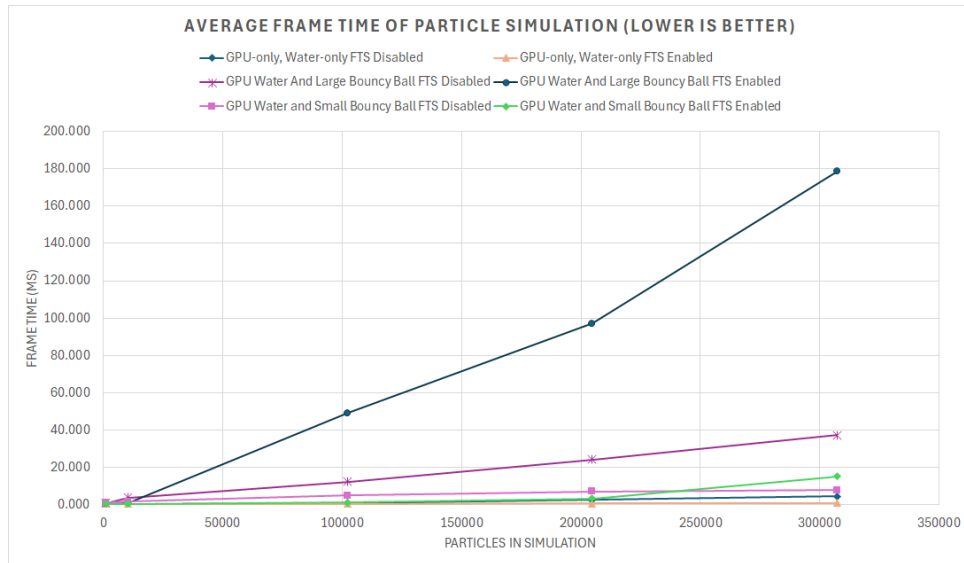
Performance Overview

All performance measurement data can be found in *Table A-1*.

There were four different hypotheses that were investigated when analyzing the project design.

1. Simulating particles of significantly different size reduces simulation performance.
2. Increasing particle count further reduces performance with mismatched particle sizes.
3. Splitting the simulation into two groups allows for a more optimal subdivision cell size, which increases performance.
4. Splitting a simulation with a uniform particle size decreases performance.

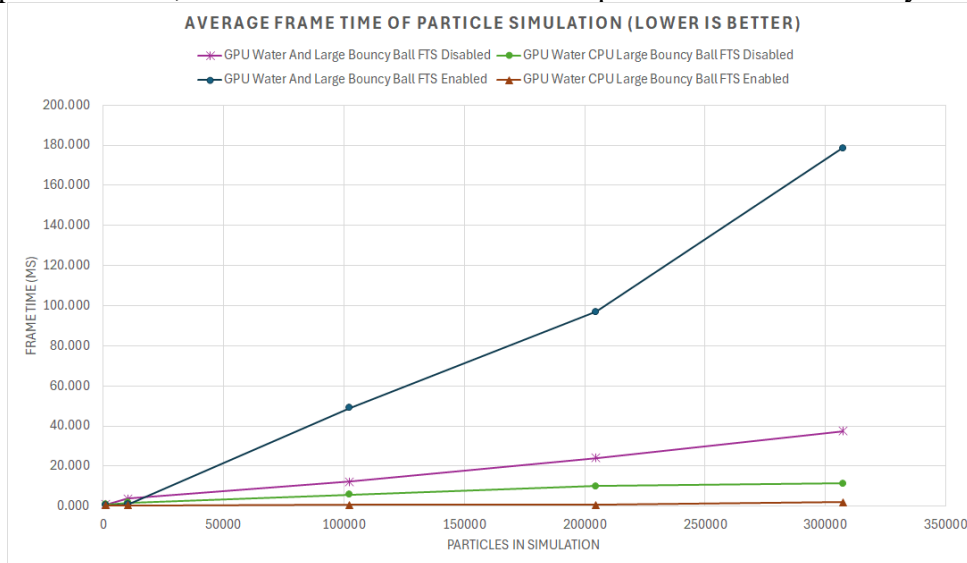
The first and second hypotheses were investigated by comparing test scenarios 1 and 2 with test scenarios 4-6.



Graph 1: GPU only comparison of single-fluid vs multi-fluid sims with particles of varying sizes.

Graph 1 demonstrates the performance loss when simulating particles of significantly different size. The water only simulations showed the best performance across all particle counts, with performance reductions visible when either the *small bouncy ball* or *large bouncy ball* fluid types are added to the water simulation. The performance impact of fixed-time steps is also seen, particularly with the *large bouncy ball* and *water* particle simulation. The average performance reduction caused by mismatched particle sizes was 83.27%. Test scenarios 1, 2, 5, and 6 represent the baseline measurements that were used when evaluating the performance of *particle-based subdivision*.

To test hypothesis three, test scenarios 7 and 8 were compared with the GPU-only baselines.

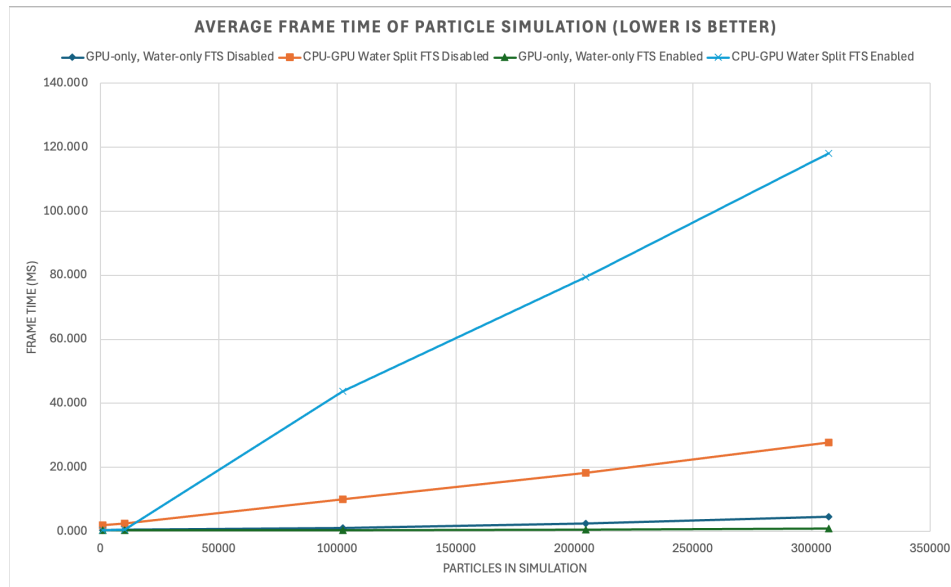


Graph 2: Performance between GPU-only multi-fluid simulation vs CPU-GPU multi-fluid simulation.

Graph 2 demonstrates the performance uplift gained as a result of *particle-based subdivision*. A significant reduction in frame time can be seen across all particle counts,

demonstrating that the performance increase is scalable to large particle counts. The average performance uplift from *particle-based subdivision* was 2929.7%. However, it should be noted that this result represents a best case scenario for performance uplift from *particle-based subdivision*.

Next hypothesis four was investigated by comparing test scenarios 1 and 2 with test scenarios 9 and 10.



Graph 3: Comparison of GPU-only single-fluid simulation vs CPU-GPU single-fluid using previous density-based subdivision algorithm.

Splitting a single fluid across the CPU and GPU requires significant coordination in order to ensure both processing elements perform useful work. **Graph 3** relates to the *density-based subdivision* alternate design mentioned earlier in this report. **Graph 3** demonstrates the performance loss when a fluid simulation with only one fluid type (in this case water) is split across the CPU and GPU. A fluid simulation of just one fluid allows for an optimal uniform spatial subdivision grid. Splitting the simulation of a single fluid across the CPU and GPU requires two particle data transfers between the CPU and GPU which introduce a significant data transfer time that hinders performance significantly. The average performance loss from splitting a single fluid type across both CPU and GPU was 83%. The poor performance of *density-based subdivision* was the primary reason it was abandoned for *particle-based subdivision*, which showed significantly better results.

Wavefront Performance

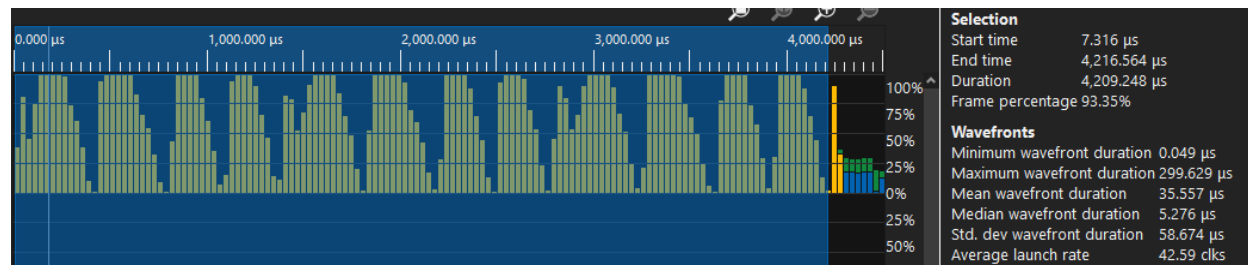


Figure 26: Wavefront analysis of scenario 7 fluid simulation for a single frame.

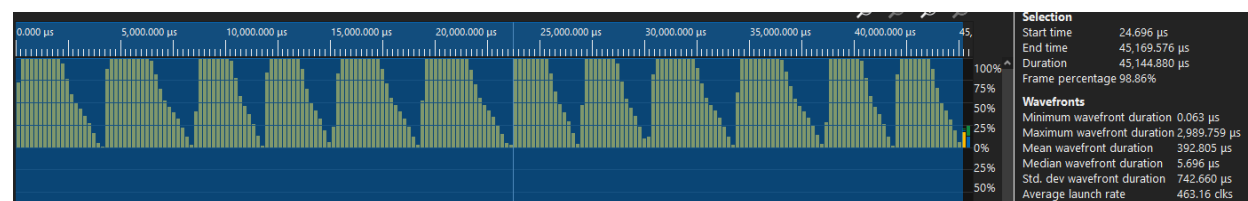


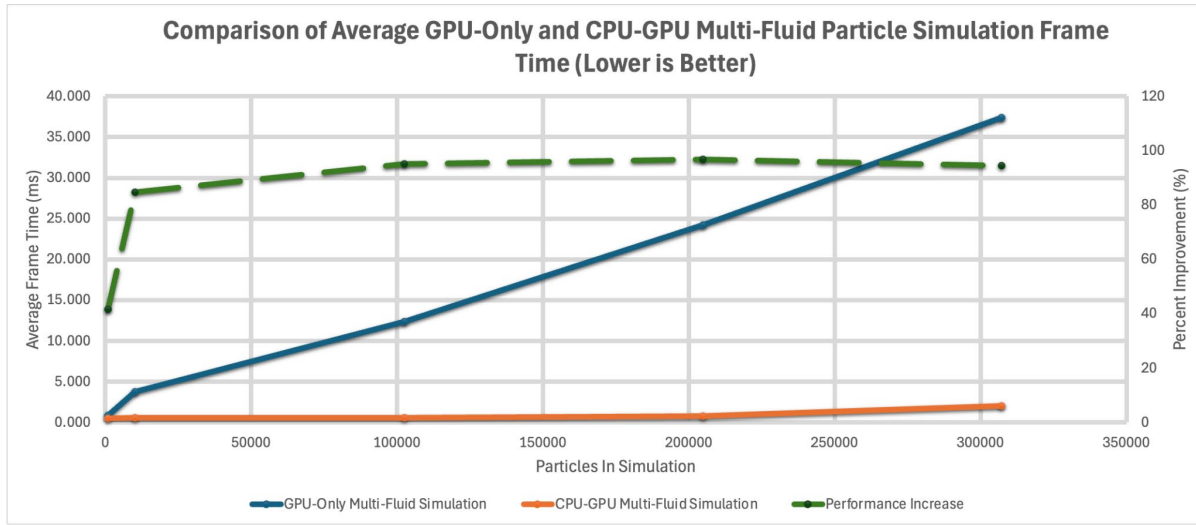
Figure 27: Wavefront analysis of scenario 5 fluid simulation for a single frame.

Table 2: Comparison of Wavefront Analysis Metrics of Test Scenario 5 and Test Scenario 7.

Wavefront Analysis Metrics:	Scenario 5 (GPU-Only):	Scenario 7 (CPU-GPU):
Total Duration:	45144.880 μs	4209.248 μs
Minimum Wavefront Duration:	0.063 μs	0.049 μs
Maximum Wavefront Duration:	2989.759 μs	299.629 μs
Mean Wavefront Duration:	392.805 μs	35.557 μs
Median Wavefront Duration:	5.696 μs	5.276 μs
Standard Deviation Wavefront Duration:	742.660 μs	58.674 μs
Average Launch Rate:	463.16 clks	42.56 clks

The wavefront comparison shown in **Table 2** demonstrates the coherency improvement obtained through *particle-based subdivision*. While *density-based subdivision* was unable to achieve performance boosts due to a memory transfer bottleneck as the "most dense" areas change every frame forcing a readback of particles from the GPU, the particle-based subdivision achieved a significant performance boost as it was not limited by memory transfers.

Analysis



Graph 4: Performance improvement of particle-based subdivision compared to a GPU-only baseline.

Comparing the best GPU-only multi-fluid performance with the best CPU-GPU multi-fluid performance, we find that the average performance increase from particle-based subdivision was 82%, a significant improvement. Our broad-phase collision scheme effectively leverages the constant-time lookup and maintenance of uniform spatial hashing while mitigating the downsides of mismatched particle sizes by using multiple uniform hashes to simulate multiple particle sizes. By splitting particles into size-based groups, we avoid the inefficiencies of a single fixed grid when applied to varied particle sizes by reducing false positives, thus improving cache coherency.

In contrast to binary tree or quadtree/octree approaches, which require $O(\log n)$ operations for insertion, deletion, and queries [11], our spatial hashing implementation avoids hierarchical traversal and rebalancing overhead. This makes our method faster for dynamic scenes and simpler to implement, enabling a simpler uniform grid approach to be applied to multi-particle collision detection schemes where collider sizes vary.

A clear limitation to the approach is that performance gains will begin to fall apart as the number of particle sizes increases beyond the number of available devices for dispatching compute calls, as particle size mismatches will degrade performance if executed on the same device, similar to the GPU-only case outlined above. A simple solution involves simply grouping similarly-sized particles into discrete particle groups for dispatching on both devices, however such a solution is limited in the event that more than 2 discrete groups are required for better performance. Additional investigation into optimal particle size ranges for discrete particle groups would be required to find the expected performance for n different particle sizes in a broader set of scenarios, as well as for implementing such a solution on more than 2 devices (eg. multiple CPU/GPU scenarios).

Conclusion

The development and testing of a heterogeneous CPU-GPU collision detection system for SPH fluid simulation has demonstrated that intelligently partitioning computational workloads can lead to significant performance improvements in real-time applications. Through the implementation of the Particle-Based Subdivision algorithm, we effectively mitigated performance bottlenecks encountered in GPU-only systems—particularly those stemming from particle size variance and inefficient broad-phase collision grid scaling.

Key to our success was the ability to offload large-particle simulations to the CPU while maintaining smaller, more parallelizable computations on the GPU. This allowed for better spatial locality, improved wavefront coherence, and reduced redundant narrow-phase collision checks, ultimately leading to more efficient frame execution. The integration of kernel-level optimizations—such as unifying physics calculations into a single kernel, and adopting counting sort for spatial subdivision—further reinforced the performance and scalability of the simulation, especially across varying hardware configurations.

Our results validate the central hypothesis: simulations that involve fluids with diverse particle sizes benefit substantially from heterogeneous computing. Performance metrics consistently showed that separating workloads by particle size enabled better use of hardware resources, delivering up to 80% improvements in frame time over baseline GPU-only implementations. Conversely, in simulations where particle sizes were uniform, heterogeneous processing did not yield substantial benefits, and in some cases introduced marginal overhead due to CPU-GPU synchronization costs.

Beyond performance gains, this project has delivered a highly interactive and visually compelling simulation environment, complete with fully functional game levels that showcase the technical features in engaging contexts. The simulation supports a wide range of fluid types and behaviors, enabling creative and educational gameplay scenarios—from casting swords and controlling drink deliveries to boiling water in puzzle-based environments.

In conclusion, this project demonstrates that a well-designed heterogeneous system can not only enhance simulation performance but also support complex, scalable, and interactive applications. Future work could explore dynamic load balancing, adaptive subdivision techniques, and deeper integration with game logic systems to further refine performance. The knowledge and system architecture developed through this capstone provide a strong foundation for ongoing research and development in real-time physics simulation and hybrid computing systems in game design.

References

- [1] M. Müller, D. Charypar, and M. Gross, "Particle-based fluid simulation for interactive applications," in *Proc. ACM SIGGRAPH/Eurographics Symp. Computer Animation*, San Diego, CA, USA, Jul. 2003, pp. 154–159. [Online]. Available: <https://matthias-research.github.io/pages/publications/sca03.pdf>
- [2] R. Salama and M. Elsayed, "A live comparison between Unity and Unreal Game Engines," *Global Journal of Information Technology: Emerging Technologies*, vol. 11, no. 1, pp. 1–7, Apr. 2021, doi: 10.18844/gjit.v11i1.5288.
- [3] A. Söderström, "A qualitative comparison study between common GPGPU frameworks," Jun. 2018. [Online]. Available: <https://liu.diva-portal.org/smash/get/diva2:1239545/FULLTEXT01.pdf>
- [4] T. M. Aamodt, W. W. L. Fung, and T. G. Rogers, *General-Purpose Graphics Processor Architectures*. Cham, Switzerland: Springer, 2018. [Online]. Available: <https://picture.iczhiku.com/resource/eetop/WHKdgQhRWGRojBxB.pdf>
- [5] D. Koschier, J. Bender, B. Solenthaler, and M. Teschner, "Smoothed particle hydrodynamics techniques for the physics based simulation of fluids and solids," 2019. [Online]. Available: https://sph-tutorial.physics-simulation.org/pdf/SPH_Tutorial.pdf
- [6] Y. R. Serpa and M. A. F. Rodrigues, "Broadmark: A testing framework for broad-phase collision detection algorithms," *Comput. Graph. Forum*, vol. 39, no. 1, pp. 436–449, 2020.
- [7] S. Le Grand, "Chapter 32. Broad-phase collision detection with CUDA," *NVIDIA Developer*, 2007. [Online]. Available: <https://developer.nvidia.com/gpugems/gpugems3/part-v-physics-simulation/chapter-32-broad-phase-collision-detection-cuda>
- [8] P. Lindermann, "The Gilbert-Johnson-Keerthi distance algorithm," *Algorithms in Media Informatics*, Munich, Germany, 2009. [Online]. Available: https://www.medien.ifi.lmu.de/lehre/ss10/ps/Ausarbeitung_Bispiel.pdf
- [9] T. Larsson, "Fast and tight fitting bounding spheres," Linköping, Sweden, 2008. [Online]. Available: <https://ep.liu.se/ecp/034/009/ecp083409.pdf>
- [10] C. Liang and X. Liu, "The research of collision detection algorithm based on separating axis theorem," *Int. J. Sci.*, vol. 2, no. 10, pp. 1–7, 2015.
- [11] B. Li and R. Mukundan, "A comparative analysis of spatial partitioning methods for large-scale, real-time crowd simulation," in *21st Int. Conf. Comput. Graph., Vis. Comput. Vis.*, 2013. [Online]. Available: <https://naos-be.zcu.cz/server/api/core/bitstreams/4bdc15e9-7743-4aa7-a13e-6d998655a766/content>
- [12] *GPU Gems 3*, ch. 46, "Bitonic sort visualization and analysis." NVIDIA Corporation, 2007.

- [13] "Sorting network for the bitonic sort," *ResearchGate*, 2020. [Online]. Available: <https://www.researchgate.net/publication/342402983/figure/fig1/AS:906061875798017@1593033393665/Sorting-network-for-the-bitonic-sort-Each-horizontal-line-represents-an-input-and-each.png>
- [14] D. Merrill and A. Grimshaw, "High performance and scalable radix sorting," in *Proc. 16th ACM Symp. Princ. Pract. Parallel Program.*, 2011, pp. 277–288.
- [15] S. Sengupta, M. Harris, Y. Zhang, and J. D. Owens, "Scan primitives for GPU computing," in *Proc. Graph. Hardw.*, 2007, pp. 97–106.
- [16] "Parallel sorting on GPU," *Medium*, 2021. [Online]. Available: https://miro.medium.com/v2/resize:fit:1400/1*uiuxN9SyZJPcIT22uTUcMw.png
- [17] D. Halbhuber, P. Schauhuber, V. Schwind, and N. Henze, "The effects of latency and in-game perspective on player performance and game experience," *Proc. ACM Hum.-Comput. Interact.*, vol. 7, no. CHI PLAY, pp. 1308–1329, Sep. 2023, doi: 10.1145/361107.
- [18] C. Ericson, *Real-Time Collision Detection*, 1st ed. Boca Raton, FL, USA: CRC Press, 2005, doi: 10.1201/b145.

Appendix

Appendix A

Table A-1: Average Frame Time (ms) for Various Fluid Simulation Test Scenarios With Respect To Particles in The Fluid Simulation.

Test Scenario*:	1	2	3	4	5	6	7	8	9	10
Fixed-Time Step (FTS):	Disabled	Enabled	Disabled	Enabled	Disabled	Enabled	Disabled	Enabled	Disabled	Enabled
Fluid Types in GPU Simulation	Water	Water	Water, Small Bouncy Ball	Water, Small Bouncy Ball	Water, Large Bouncy Ball	Water, Large Bouncy Ball	Water	Water	Water	Water
CPU Object Types in Scenario:	None	None	None	None	None	None	Large Bouncy Ball	Large Bouncy Ball	Water	Water
1024 Particles:	0.478	0.412	1.113	0.503	0.881	0.492	0.965	0.514	1.982	0.483
10240 Particles:	0.559	0.416	1.807	0.562	3.766	0.956	1.835	0.579	2.506	0.523
102400 Particles:	1.059	0.471	4.991	1.224	12.364	49.116	5.802	0.603	10.119	43.706
204800 Particles:	2.586	0.630	7.246	3.435	24.201	97.087	10.136	0.805	18.262	79.365
307200 Particles:	4.536	0.974	7.921	15.207	37.369	178.571	11.371	2.040	27.793	117.925

Test scenario details can be found in **Table 1.*